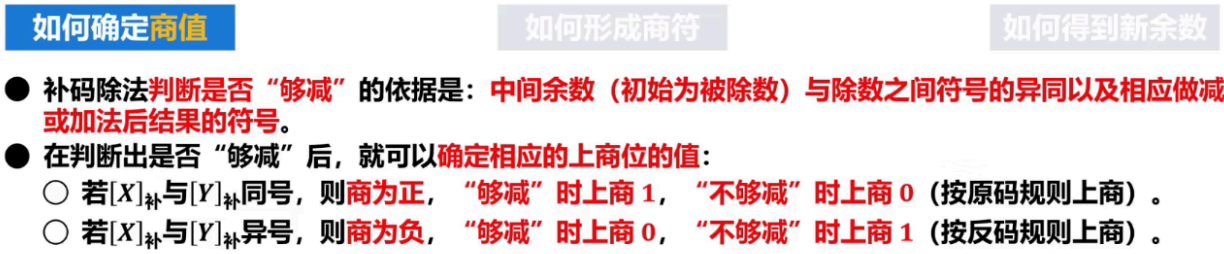
一、如何确定商值

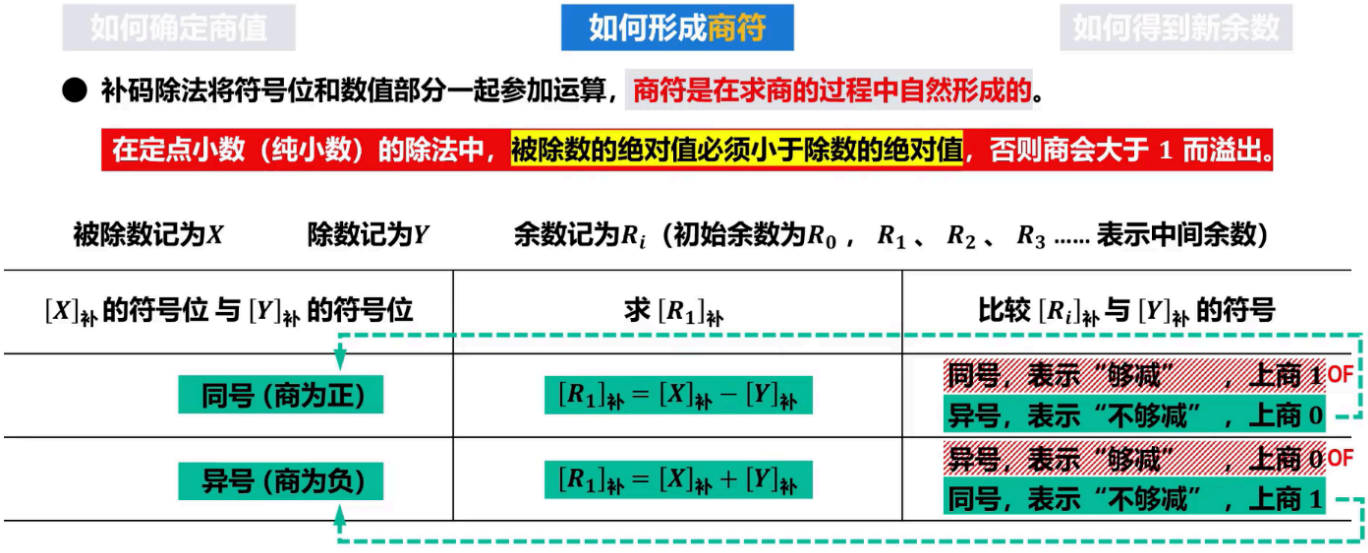


$[X]_{补}$ 的符号位 与 $[Y]_{补}$ 的符号位	求 $[R_1]_{补}$	比较 $[R_i]_{补}$ 与 $[Y]_{补}$ 的符号
同号（商为正）	$[R_1]_{补}=[X]_{补}-[Y]_{补}$	同号，表示“够减”，上商 1 异号，表示“不够减”，上商 0
异号（商为负）	$[R_1]_{补}=[X]_{补}+[Y]_{补}$	异号，表示“够减”，上商 0 同号，表示“不够减”，上商 1

注释:

- X、Y同号：当差的符号与Y相异（也与X相异），证明减Y使得X逆转了，显然这里Y的绝对值更大，所以不够“减”
- X、Y异号：当差的符号与Y相同时，证明Y对X的削弱程度，超过了X本身，直接把X的符号反转了，所以Y的绝对值更大，即不够“减”

二、如何确定商符



注：不可误以为定点整数的除法没有溢出现象，只是因为定点小数溢出现象比较特殊，这里才重点提到

关于定点整数除法的溢出:

定点整数除法通常约定：

- 被除数为 **双字长**（2n 位）
- 除数为 **单字长**（n 位）
- 商和余数也取 **单字长**（n 位）

此时溢出的判断逻辑是：

- 若被除数的高 n 位（即高位部分）的绝对值 ≥ 除数的绝对值，则商的有效位数将超过 n 位
- 单字长无法容纳该商，**同样发生溢出**

例如，用 8 位除数去除 16 位被除数，若被除数高 8 位已经大于除数，那么商至少需要 9 位才能表示，8 位存储单元放不下，即溢出

三、如何确定新余数

补码除法运算（不恢复余数法）

被除数记为X　　除数记为Y　　余数记为R_i（初始余数为R_0，R_1、R_2、R_3 ..... 表示中间余数）

$[X]_{补}$与$[Y]_{补}$	商符	$[R_1]_{补}$	$[R_i]_{补}$与$[Y]_{补}$	上商位	$[R_{i+1}]_{补}$
同号	正	$[R_1]_{补}=[X]_{补}-[Y]_{补}$	同号，表示“够减”	1	$[R_{i+1}]_{补}=2[R_i]_{补}+[-Y]_{补}$
			异号，表示“不够减”	0	$[R_{i+1}]_{补}=2[R_i]_{补}+[Y]_{补}$
异号	负	$[R_1]_{补}=[X]_{补}+[Y]_{补}$	同号，表示“不够减”	1	$[R_{i+1}]_{补}=2[R_i]_{补}+[-Y]_{补}$
			异号，表示“够减”	0	$[R_{i+1}]_{补}=2[R_i]_{补}+[Y]_{补}$

如果对商的精度没有特殊要求，一般可采用“末位恒置 1”法，这种方法操作简单，易于实现，而且最大误差仅为2^{-n}。

四、示例

补码除法运算（不恢复余数法）

【举例】已知 $x=-0.1001$，$y=+0.1101$，求 $[x]_{补}\div[y]_{补}$。

解析：　$[x]_{补}=1.0111$，　$[y]_{补}=0.1101$，　$[-y]_{补}=1.0011$，

	余数 $[R]_{补}$	商 $[Q]_{补}$	
	1.0111	0.0000	初始余数 $[R_0]_{补}$ 是 $[x]_{补}$
$+[y]_{补}$	0.1101		由于 $[x]_{补}$ 与 $[y]_{补}$ 异号，因此 $[x]_{补}+[y]_{补}$
	0.0100	0.0001	$[R_1]_{补}$ 与 $[y]_{补}$ 同号，上商 1
	0.1000	0.001	$\ll 1$
$+[-y]_{补}$	1.0011		$+[-y]_{补}$
	1.1011	0.0010	$[R_2]_{补}$ 与 $[y]_{补}$ 异号，上商 0
	1.0110	0.010	$\ll 1$
$+[y]_{补}$	0.1101		$+[y]_{补}$
	0.0011	0.0101	$[R_3]_{补}$ 与 $[y]_{补}$ 同号，上商 1
	0.0110	0.101	$\ll 1$
$+[-y]_{补}$	1.0011		$+[-y]_{补}$
	1.1001	0.1010	$[R_4]_{补}$ 与 $[y]_{补}$ 异号，上商 0
	1.0010	1.0100	$\ll 1$，末位商恒置1

n 位定点小数（本例中 $n=4$）补码除法：

- 上商共 $n+1$ 次（包括末位恒置1），首次上商可用来判断是否溢出；
- 左移一位共 n 次，可用移位次数判断除法是否结束。

五、一些疑问

上面这些讲的都是定点小数的解题流程，但定点小数和定点整数的除法过程是有一定区别的，我在这里做总结：

一句话总纲：加减交替法的**内核完全相同**（第一步定商符、n 轮“左移—加减—上商”、符号位全程参与运算），两类题的

差异**只集中在三个环节**——初始装数、移位的含义、结果的组装与解读

5.1 一表看懂两类除法（n+1 位机器，n=4 即 5 位）

维度	定点小数（图中模板）	定点整数（−8÷2 模板）
题目特征	x = ±0.xxxx，且 **|x| < |y|**	x 为任意整数（如 −8）
被除数装哪	整个 [x]补 放进 **R**	符号扩展到 2n 位后**拆两半**
R 初值	[x]补（如 1.0111）	扩展的符号位（如 111111）
Q 初值	**0.0000**	被除数的低 n 位（如 1000）
联合寄存器	R 本身就是被除数	**R||Q 拼起来**才是被除数
移位含义	空位补 0，商位逐个“长”进 Q	被除数位从 Q **落入** R，商位**顶**入 Q
商的组装	Q 最终整体 = 商（含符号位）	**商符(第1步) || Q 的 n 位**
真余数	R 终值 **×2^{-n}**	R 终值（必要时修正）
溢出条件	|x| ≥ |y|（商≥1）	商超出 n+1 位补码范围
步数	相同：**n+1 次加减、n 次移位**	相同
### 5.2 统一五步内核（唯一分叉在“装数”）		
1. 求 [y]补、[−y]补；**R 建议用双符号位**（图中单符号位写法在该例数值范围内也成立，但双符号位更保险）		
2. 装被除数 → **小数：R=[x]补，Q=0**；**整数：R=符号扩展位，Q=低 n 位**（R||Q=被除数）		
3. **第一步不移位**：x、y 同号 → +[−y]补，异号 → +[y]补；比较 R 与 y 符号**上商符**		
4. 重复 n 轮：**{R,Q} 联合左移 1 位** → 按**上一次**的商选操作（商1→+[−y]补，商0→+[y]补）→ R 与 y **同号商 1、异号商 0**		
5. 收尾：**末位恒置 1**（误差不超末位 1，408 默认可接受）；若要求精确余数且末位商为 0 → R +[y]补 修正		
### 5.3 两个模板卡（考场上照抄框架）		
小数模板（即你给的图）：x = −0.1001 ÷ 0.1101		

```
R=1.0111  Q=0.0000          ←被除数整体在R
+[y]补 → 0.0100  商符1
  4轮：左移→按上商加减→上商
商 = 1.0101（末位恒置1）
真余数 = 1.1111 × 2⁻⁴ = -0.0001
验算：-0.1011×0.1101 + (-0.0001) = -0.1001 ✓
```

整数模板：x = −8 ÷ 2

```
R=1.1111  Q=1000            ←R‖Q = -8
+[y]补 → 0.0001  商符1
  4轮：左移(被除数落位)→加减→上商 → Q=1100
商 = 商符1 ‖ 1100 = 11100 = -4
余数：末位商0 → R+[y]补 = 0.0000
验算：-4×2 + 0 = -8 ✓
```

5.4 拿到题先问三个问题（判断流程）

1. **数里有没有小数点？** x=±0.bbb → 小数模板；x=±整数 → 整数模板
2. **是整数题？先做符号扩展拆分**（最容易丢分的一步：Q 清零 = 把题目改成 −128，全盘皆错）
3. **要求精确解还是可恒置1？** 未说明 → 恒置1；要验算 x=qy+r → 用修正法

5.5 高频选择题考点（背结论）

- 补码加减交替法：**n+1 次加减、n 次移位、商共 n+1 位**（首位为商符）
- **第一步不移位**，上的第一位就是商符——符号位参与运算的直接体现
- 上商规则：部分余数与除数**同号商 1**，异号商 0；本步商决定下一步加还是减
- 小数除法前提 **|x| < |y|**（否则商 ≥1 溢出）；整数除法溢出看商是否超范围
- 恒置 1 的误差**不超过末位 1**（小数即 2^{-n}）
- 合法结果必满足：**|r| < |y|，且 r 与 x 同号（r=0 除外）**
- 双符号位的作用：容纳部分余数、判断溢出

5.6 大题防错清单

陷阱	后果
整数题 Q 清零	联合寄存器≠被除数，商符错、中间溢出
小数题把 x 拆进 Q	无高位段可落，商没有来源
忘记第一步不移位	商符错位，全表报废
按当次（而非上次）商选加减	累进错误，后面全错
5 位结果直接写、没丢第 6 位进位	补码按模运算，必须丢弃
验算时小数余数忘乘 2^{-n}	验算"失败"，其实算对了
万能验算（每题必做，10 秒保命）：	
$q \times y + r = x, \quad \lvert r\rvert < \lvert y\rvert, \quad r$ 与 x 同号或为 0	

六、以-8除以2讲解整数补码除法的具体细节

```
                余数[R]补       商[Q]补
+[y]补          1.1111        1000      ←R=符号扩展，Q=被除数低4位
                0.0010
                0.0001        1000      ←上商符【1】✓
+[-y]补         0.0011        000
                1.1110
                0.0001        0001      ←【1】
+[-y]补         0.0010        001
                1.1110
                0.0000        0011      ←【1】
+[-y]补         0.0000        011
                1.1110
                1.1110        0110      ←【0】
+[y]补          1.1100        110
                0.0010
                1.1110        1100      ←【0】
+[y]补          1.1110                  ←末位商0，余数修正
                0.0010
                0.0000        1.1100
```

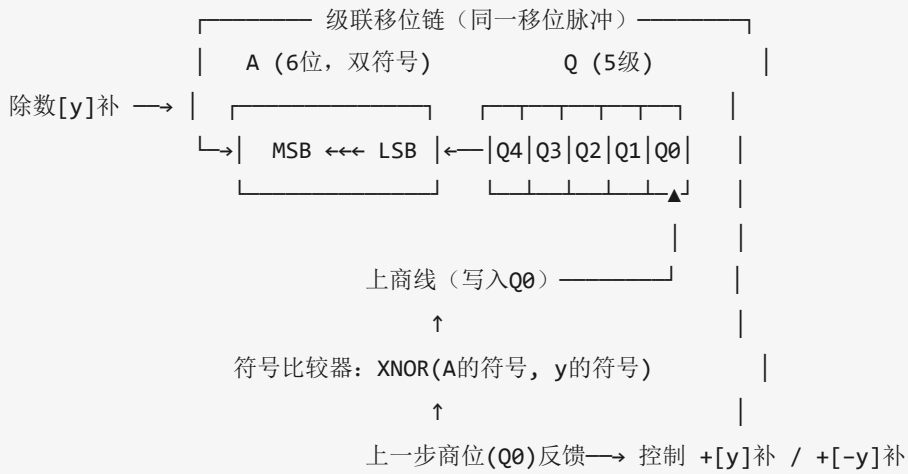
商=1.1100=−4,余数=0.0000=0,(−4)×2+0=−8✓商=1.1100=−4,余数=0.0000=0,(−4)×2+0=−8✓

表格里有一个疑问:
第一步产生的商符，没法写到Q里，因为Q是四位，每一位都是除数的数值位，没有位置让第一步的商符上位，所以实际上在硬件水平，Q为五位:

硬件视角：商符其实“有地方写”——Q 本来就是 5 级移位寄存器

真实硬件里不存在“无处可写、事后挂账”。教科书表格把 Q 画成 4 格，只是排版省地方；实际数据通路中 **Q 是 n+1 = 5 级的移位寄存器**，比表格多出一格。商符在这一格写入后，**靠 4 次左移自然“走”到最高位**——不需要任何事后拼接

一、数据通路结构（文字版框图）



关键三点：

1. **上商线固定接在 Q 的最低级 Q0**——移位寄存器的“装入端”天然在低位，逻辑上最便宜
2. **商位由比较器组合逻辑产生**：R 与 y 同号输出 1、异号输出 0，每个周期时钟打入 Q0
3. **加/减选择由上一步的商位反馈控制**（第 1 拍无反馈，由 x、y 初始符号比较决定）

二、用 −8÷2 逐拍跟踪 Q（5 格全画出来）

装入：A = 111111，Q = 1000**0**（低 4 位放被除数，Q0 是空余级，恒 0）

拍	左移后 A	本拍加减	加减后 A	上商→打入 Q0	Q（Q4 Q3 Q2 Q1 Q0）	商符位置
装入	—	—	111111	—	1 0 0 0 **0**	—
1	(不移位)	+[y]补	000001	**1（商符）**	1 0 0 0 **1**	Q0
2	000011	+[−y]补	000001	1	0 0 0 1 **1**	Q1
3	000010	+[−y]补	000000	1	0 0 1 1 **1**	Q2
4	000000	+[−y]补	111110	0	0 1 1 1 **0**	Q3
5	111100	+[y]补	111110	0	1 1 1 0 **0**	**Q4（符号位）**
收尾	末位商0 → 余数+[y]补修正		000000		1 1 1 0 0	定格

盯着"商符"那一列看：它第 1 拍写入 Q0，之后每拍随链左移一级，4 拍后正好走到 Q4——**Q 的最高位就是符号位。**终值 Q = 11100 = −4，无需任何拼接操作

同时注意被除数的消化：Q4~Q1 里的 1,0,0,0 在 4 次左移中逐拍落入 A（第 2~5 拍各消化 1 位），恰好腾出的 4 个坑由

后 4 位商补上——**一条移位链同时完成"被除数上供"和"商位下填"**，这就是为什么硬件只肯给 Q 多留一级、而不愿加一条"往 MSB 写数"的专用通路

三、另一种等价实现：专用商符触发器

有些简单设计确实用"挂账"方案，但物理形态是一个 **1 位商符触发器**：
- 第 1 拍比较结果打入商符 FF（对补码除法）；或者更直接：**商符 = sign(x) ⊕ sign(y)**，用一个异或门在除法开始前就算好
- 结束时，商总线的高位由一根**固定连线**接商符 FF，低 n 位接 Q——所谓"拼在前面"，在硬件里就是**布线**，零成本、零拍数

这与原码除法互为镜像：原码除法硬件里商符本来就是一个由异或门驱动的独立触发器，尾数单独算；补码除法的"第一次上商即商符"只是把这件事并进了统一的移位链

四、统一视角：整数、小数用的是同一条数据通路

	A 装入	**Q(5级) 装入**
定点小数 x=−0.1011	[x]补 扩双符号：11 0111→110111	00000（全空，商符第1拍进Q0）
定点整数 x=−8	符号扩展：111111	1000**0**（低4位被除数+空余级0）
控制逻辑、移位链、比较器、拍数完全相同，唯一的差别是装入常数。 小数题表格里你能"看见"商符在 Q 里移动，是因为 Q 里的 0 是无害的；整数题表格把 Q 截成 4 格展示，商符才显得"没地方去"——硬件里它一直都在 Q 里，只是占了那格没画出来的第 5 级		
#### 五、一点延伸与应试落点		
- 现代 CPU 用基数 4 的 SRT 等算法，商位（数字）由部分余数最高位查表选出，商寄存器同样预置了符号位的位置（on-the-fly 转换逐拍拼装商）——思想一脉相承：**给结果留足级数，让移位把每一位送到家。**此内容 408 不考		
- **408 只要求到"商符由第一次上商产生（补码）/ 两数符号异或（原码）"这一层。**但理解本页的移位链后，你之前三个疑问（为什么 R 是符号扩展、为什么 Q 不清零、商符写到哪）会统一成一个图景：**A||Q 是一条 11 级级联移位链，被除数从左端供位、商从右端填位，商符只是第一个填入的位**		